

Algoritmi e Strutture Dati – Parte A – 12/06/2020

Esercizio A1, punti ≥ 8

Trovare i limiti superiori e inferiori più stretti possibili per la seguente equazione di ricorrenza, utilizzando il metodo di sostituzione (detto anche per tentativi):

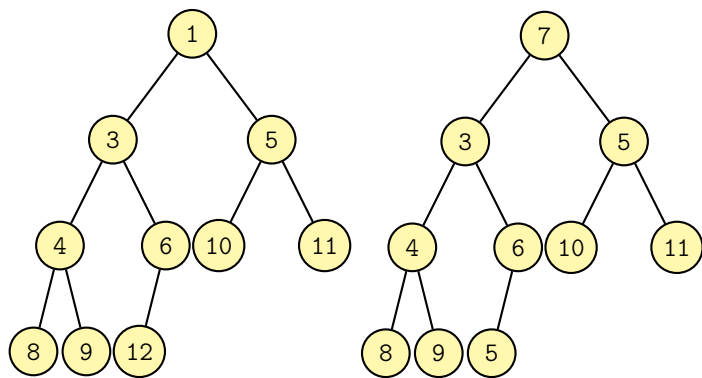
$$T(n) = \begin{cases} T(\lfloor \frac{n}{2} \rfloor) + 2T(\lfloor \frac{n}{2\sqrt{2}} \rfloor) + 7T(\lfloor \frac{n}{4} \rfloor) + n^2 & n \geq 4 \\ 1 & n < 4 \end{cases}$$

Esercizio A2, punti ≥ 10

Scrivere un algoritmo **boolean** `isMinHeap(TREE T)` che prenda in input un albero binario completo composto da n nodi contenenti valori interi nel campo *value* e restituisca **true** se l'albero rispetta le regole dei Min Heap, **false** altrimenti.

Discutere correttezza e complessità computazionale dell'algoritmo proposto.

Nella figura seguente, l'albero di sinistra è un albero completo che rispetta le regole di minheap, l'albero di destra non le rispetta.



Esercizio A3 – Mystery array – Punti ≥ 12

Un *mystery array* è una struttura dati di tipo sequenza, che permette di accedere ai suoi elementi tramite indici. Tuttavia, non ha primitive per ottenere il numero di elementi in esso contenuti. Se si tenta di leggere un indice i presente in un mystery array A , viene restituito il valore $A[i]$ corrispondente. Se i è superiore alla dimensione del vettore, viene restituito il valore speciale $\perp$.

Esempio: Sia $A = [-5, -3, 11, 13, 11]$ un mystery array di 5 elementi. $A[4]$ restituisce 13, mentre $A[1000]$ restituisce $\perp$.

Scrivere un algoritmo

int countNegatives(MISTERYARRAY A)

che prenda in input un mystery array non vuoto, ordinato A contenente interi, e restituisca il numero di valori negativi presenti nel mystery array.

Spiegare il funzionamento e discutere la complessità computazionale dell'algoritmo proposto. Algoritmi di complessità $\Theta(n)$ o superiore, dove n è l'effettiva dimensione del mystery array, non verranno presi in considerazione.

Algoritmi e Strutture Dati – Parte B - 12/06/2020**Esercizio B1, punti ≥ 8**

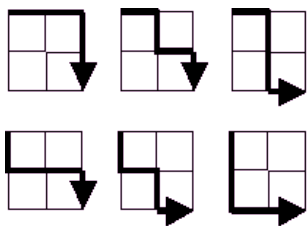
Si consideri una scacchiera di dimensione $2 \times n$ (2 righe, n colonne), rappresentata da una matrice di interi positivi A , dove $A[i][j]$ rappresenta il guadagno della casella (i, j) .

Scrivere un algoritmo `int maxGain(int[][] A, int n)` che restituisca il guadagno massimo che si può ottenere dalla somma di un sottoinsieme di caselle, in modo tale che non esistano due caselle del sottoinsieme che siano adiacenti orizzontalmente, verticalmente o diagonalmente.

Discutere correttezza e complessità dell'algoritmo proposto.

Esercizio B2, punti ≥ 10

Partendo dall'angolo in alto a sinistra di una griglia 2×2 , e muovendosi solo a destra o in basso, ci sono esattamente 6 percorsi distinti per raggiungere il bordo in basso a destra.



Scrivere un algoritmo `int paths(int n)` che prenda in input un intero positivo n e restituisca il numero di percorsi distinti per andare dall'angolo in alto a sinistra all'angolo in basso a destra di una griglia $n \times n$, muovendosi solo a destra e in basso.

Discutere correttezza e complessità dell'algoritmo proposto.

Esercizio B3, punti ≥ 12

Dato un intero n , una *somma palindroma* di n è una sequenza palindroma di k valori $a_1, \dots, a_k$ tale che $\sum_{i=1}^k a_i = n$.

Ad esempio, il numero 6 può essere scritto come somma palindroma esattamente in 8 modi diversi:

$$(1, 1, 1, 1, 1, 1), (1, 1, 2, 1, 1), (1, 2, 2, 1), (1, 4, 1), (2, 1, 1, 2), (2, 2, 2), (3, 3), (6)$$

Scrivere un algoritmo `generate(int n)` che prenda in input un valore n e stampi tutte le somme palindrome di n . Notate che in questo esercizio, si stampano tutte le permutazioni. Ad esempio, $(1, 2, 2, 1)$, $(2, 1, 1, 2)$ sono permutazioni, ma vanno generate entrambe.

Se volete, potete risolvere una versione semplificata del problema, che stampa solo somme palindrome di lunghezza pari, per l'80% del punteggio. È anche un buon modo per iniziare a risolvere il problema generale.